CEN3024C Module 5 Project

For this project, we will extend the Module 5 Task Manager application by implementing Swagger to document and interact with our REST APIs. Instead of relying on cURL commands, the Swagger UI will be used to explore, execute, and validate API endpoints in order to gain experience with integrating API documentation tools into a Spring Boot application and provide a more user-friendly way to test functionality. Note that this project will build on the entities and associations implemented in Module 4, no additional domain logic will be needed.

Objectives

- Integrate Swagger into the Spring Boot Task Manager application.
- Generate OpenAPI documentation automatically from your controllers
- Use Swagger UI to test and validate endpoints
- Verify that DTO request/response models are accurately represented
- Replace cURL-based testing with Swagger-based validation
- Prepare to implement the same documentation standards in your team project

Use the project in the assignment repository to complete the following tasks.

NOTE: be sure to configure a postgres data connection to the taskmanager_db using the Database tool so the data insertion script runs successfully at startup. This script will populate the database with sample data.

1. **Integrate Swagger into the Project**
    - Note the additional dependencies in the POM file for Spring Doc and Bean validation.
    - Start the server and confirm /v3/api-docs is accessible via http://localhost:8080/v3/api-docs (this should display raw JSON)
    - Confirm /swagger-ui/index.html loads successfully via http://localhost:8080/swagger-ui/index.html.
    - Gaze in wonder at the colorful documentation of the APIs and revel in the realization that cURL will no longer be necessary.
2. **Generate API Documentation**
    - Verify that Swagger UI correctly lists endpoints for Tasks, Users, Subtasks, and Priorities.
    - Confirm that DTOs are represented accurately in the request/response models and that validation constraints (e.g., @NotNull, @Size) appear in documentation (these are shown under “Schemas” in the Swagger UI).
3. **Explore and Test Endpoints with Swagger UI**
    - Use the Swagger interface to execute the CRUD operations for Users, Tasks, and Subtasks
    - Validate associations (e.g., assign a User to a Task, add a Subtask to a Task) directly from Swagger.
    - Confirm that API responses display the expected DTO fields.

4. **Complete Regression Tests**
    - Confirm that entities and associations implemented in Module 4 (Users, Subtasks, Priorities, Tasks) still function correctly.
    - Use Swagger-based tests as regression checks to ensure persistence, relationships, and DTO mappings remain intact after integration.

API Call Sequence Checklist

Use the following sequence to demonstrate complete system functionality.

- User Endpoints
    - POST /users - create a new user (with username + password).
        - Sample Request Body:

          ```
          {
           "username": "ProfS",
           "password": "secret123"
          }
          ```
        - Note that password is not returned in response text
    - GET /users/{id} - confirm the user was created.
    - PUT /users/{id} - update user details (e.g., change username).
        - Set Parameter - User ID is 1 from previous POST
        - Sample Request Body:

          ```
          {
           "username": "profs",
           "password": "secret123"
          }
          ```
    - DELETE /users/{id} - verify deletion works.
- Task Endpoints
    - POST /tasks - create a new task (with title, description, status, dueDate).
        - Sample Request Body:

          ```
          {
           "title": "Finish Module 5 Project",
           "description": "Complete all DTOs and test with Swagger UI",
           "status": "PENDING",
           "dueDate": "2025-10-12"
          }
          ```
    - GET /tasks/{id} - confirm the task was created.
    - PUT /tasks/{id} - update task details (e.g., change status and priority).
        - Sample Request Body:

          ```
          {
           "title": "Finish Module 5 Project",
           "description": "Complete all DTOs and test with Swagger UI",
           "status": "IN_PROGRESS",
          ```

```
  "dueDate": "2025-10-12",
  "priority": { "id": 3 }
}
```

    - DELETE /tasks/{id} - verify deletion works.
- Subtask Endpoints
    - POST /subtasks - create a subtask linked to a task (via taskId).
    - GET /subtasks/{id} - confirm the subtask was created.
    - PUT /subtasks/{id} - update subtask details (e.g., mark as IN_PROGRESS).
    - DELETE /subtasks/{id} - (optional) verify deletion works.
- Associations
    - POST /tasks/{taskId}/users/{userId} - assign a user to a task.
        - No request body for this; set user ID and task ID in parameters.
    - GET /tasks/{id} - confirm the task shows assigned usernames.
    - GET /users/{id} - confirm the user shows associated task titles.
- Priority Endpoints
    - GET /priorities to verify all three priorities are returned
    - GET /priorities/{id} – select specific priority value

Submission

- Upload your forked and working project to the assignment repository (submission must be in the correct project format; do not create a new/separate project folder or zip the project).
- Provide a set of screenshots from Swagger UI that demonstrate successful testing of key endpoints (at minimum: one User, one Task with a Subtask, and the Task–User association).
- Clearly label each screenshot with the corresponding endpoint tested.